

DML (서브쿼리)

- 서브쿼리

SQL 쿼리 안에 포함된 또 다른 쿼리

• 서브쿼리

Donald 라는 직원과 같은 부서에 있는 직원 이름 조회할 때 기본적인 코드는 아래와 같이 2번에 걸쳐 SELECT를 사용해야 함

```
700 -- Donald 직원의 부서 번호
701 SELECT DEPARTMENT_ID FROM EMPLOYEES e
702 WHERE FIRST_NAME = 'Donald';
703
704 -- 부서 번호 50에 속한 직원 이름
705 SELECT FIRST_NAME FROM EMPLOYEES e
706 WHERE DEPARTMENT_ID = 50;
```

• 서브쿼리

서브쿼리를 사용할 경우 한번의 쿼리로 조회 가능

() 안에 실행할 쿼리 작성

```
706 SELECT FIRST_NAME FROM EMPLOYEES e
707 WHERE DEPARTMENT_ID = ( SELECT DEPARTMENT_ID FROM EMPLOYEES e
708 WHERE FIRST_NAME = 'Donald');
```

Results 1 Results 2 Results 3 ×
SELECT FIRST_NAME FROM EMPLOYEES e WHERE DEPART | Enter a SQL expression to filter results (use Ctrl+Space)

	FIRST_NAME
17	Michael
18	Ki
19	Hazel
20	Renske
21	Stephen

• 설명

1. 서브쿼리가 실행
2. Donald가 속한 부서의 ID를 찾음
3. 찾은 결과(50)를 DEPARTMENT_ID = 50 으로 대입되어 조건식 생성
4. 메인 쿼리의 SELECT 실행

```
706 SELECT FIRST_NAME FROM EMPLOYEES e
707 WHERE DEPARTMENT_ID = ( SELECT DEPARTMENT_ID FROM EMPLOYEES e
708                          WHERE FIRST_NAME = 'Donald');
```

Results 1 Results 2 Results 3 ×

SELECT FIRST_NAME FROM EMPLOYEES e WHERE DEPART Enter a SQL expression to filter results (use Ctrl+Space)

	FIRST_NAME
17	Michael
18	Ki
19	Hazel
20	Renske
21	Stephen

DML

(단일행 서브쿼리)

• 단일행 서브쿼리

결과값의 데이터(행)가 1개인 서브 쿼리로서 단일행 비교 연산자와 사용할 수 있음

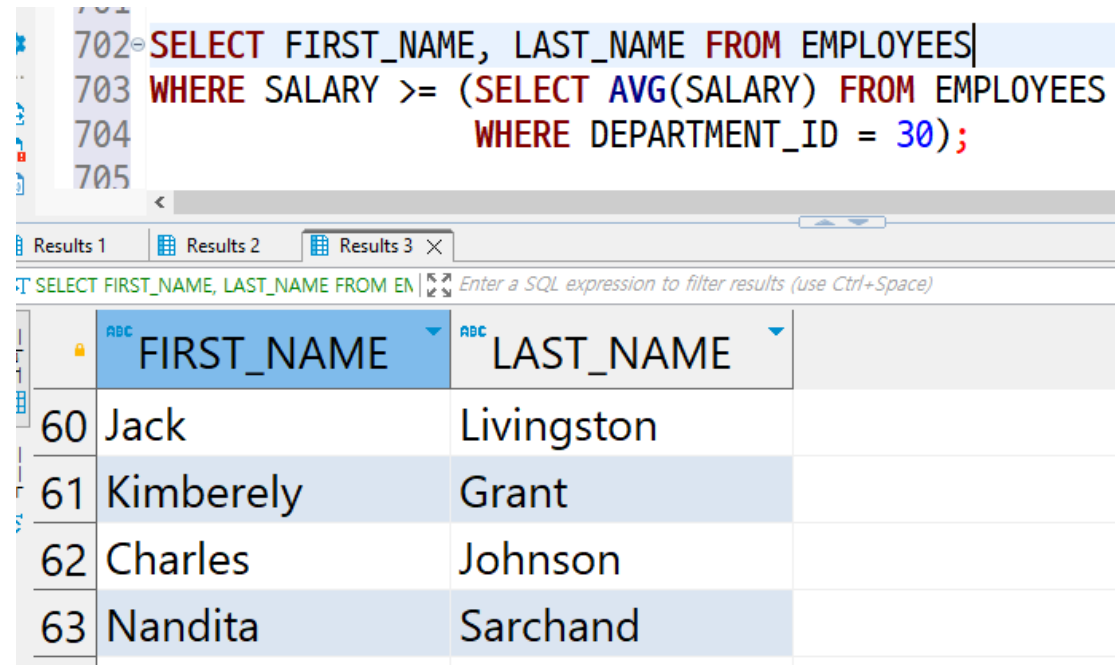
* 단일행 비교 연산자 : =, <, <=, >=, >, !=

```
702 SELECT FIRST_NAME, LAST_NAME FROM EMPLOYEES|
703 WHERE SALARY >= (SELECT AVG(SALARY) FROM EMPLOYEES
704                  WHERE DEPARTMENT_ID = 30);
705
```

	FIRST_NAME	LAST_NAME
60	Jack	Livingston
61	Kimberely	Grant
62	Charles	Johnson
63	Nandita	Sarchand

• 설명

1. 서브쿼리가 실행
2. DEPARTMENT_ID가 30인 직원들의 평균 월급을 반환
3. 찾은 평균 연봉값이 SALARY >= 조건식에 대입되어 조건식 생성
4. 메인 쿼리의 SELECT 실행



The screenshot shows a SQL IDE interface. At the top, a query is entered in a text area:

```
702 SELECT FIRST_NAME, LAST_NAME FROM EMPLOYEES  
703 WHERE SALARY >= (SELECT AVG(SALARY) FROM EMPLOYEES  
704                  WHERE DEPARTMENT_ID = 30);  
705
```

Below the query, there are tabs for 'Results 1', 'Results 2', and 'Results 3'. The 'Results 3' tab is active, displaying the results of the query. The results are shown in a table with two columns: 'FIRST_NAME' and 'LAST_NAME'. The table contains four rows of data:

	FIRST_NAME	LAST_NAME
60	Jack	Livingston
61	Kimberely	Grant
62	Charles	Johnson
63	Nandita	Sarchand

• 단일행 서브쿼리 사용해보기 - 1

가장 높은 월급을 받는 직원의 이름 조회

```
700 SELECT FIRST_NAME
701 FROM EMPLOYEES
702 WHERE SALARY = (SELECT MAX(SALARY) FROM EMPLOYEES);
```

Results 1 Results 2 Results 3 ×

SELECT FIRST_NAME FROM EMPLOYEES WHERE SALARY = (SELECT MAX(SALARY) FROM EMPLOYEES); Enter a SQL expression to filter results (use Ctrl+Space)

	ABC FIRST_NAME
1	Steven

• 단일행 서브쿼리 사용해보기 - 2

입사일이 가장 빠른 직원의 이름 조회하기

```
700 SELECT FIRST_NAME
701 FROM EMPLOYEES
702 WHERE HIRE_DATE = (SELECT MIN(HIRE_DATE)
703                     FROM EMPLOYEES);
704
```

Results 1 Results 2 Results 3 ×

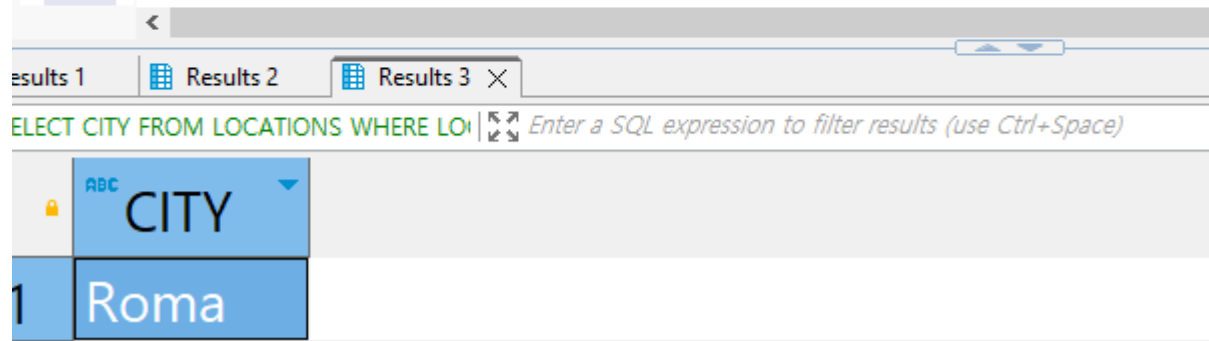
SELECT FIRST_NAME FROM EMPLOYEES WHERE HIRE_DATE = (SELECT MIN(HIRE_DATE) FROM EMPLOYEES); Enter a SQL expression to filter results (use Ctrl+Space)

	ABC FIRST_NAME
1	Lex

- 단일행 서브쿼리 사용해보기 - 3

가장 낮은 LOCATION_ID를 가진 도시 조회

```
710 SELECT CITY  
711 FROM LOCATIONS  
712 WHERE LOCATION_ID = (SELECT MIN(LOCATION_ID)  
713                        FROM LOCATIONS);  
714
```



The screenshot shows a SQL query execution interface. At the top, there are tabs for 'results 1', 'Results 2', and 'Results 3'. Below the tabs, the query text is visible: 'SELECT CITY FROM LOCATIONS WHERE LOCATION_ID = (SELECT MIN(LOCATION_ID) FROM LOCATIONS);'. The results are displayed in a table with a single row. The first column is labeled 'CITY' and the value is 'Roma'.

	CITY
1	Roma

• 단일행 서브쿼리 사용해보기 - 4

가장 높은 보너스를 받는 직원의 이름 조회

```
710 SELECT FIRST_NAME  
711 FROM EMPLOYEES  
712 WHERE COMMISSION_PCT = (SELECT MAX(COMMISSION_PCT)  
713                          FROM EMPLOYEES);  
714
```

results 1

Results 2

Results 3

Enter a SQL expression to filter results (use Ctrl+Space)

	ABC	FIRST_NAME
1		John

• Q1. 실습

1. 사수 번호(MANAGER_ID)가 가장 낮은 직원의 이름과 핸드폰 번호를 출력하세요.
2. 사번(EMPLOYEE_ID)가 가장 높은 직원의 이름과 사번을 출력하세요.
3. LOCATIONS 테이블에서 COUNTRY_ID가 가장 많은 도시(CITY)의 이름을 출력하세요.

DML

(다중행 서브쿼리)

• 다중행 서브쿼리

결과값의 데이터(행)가 여러 개인 서브 쿼리로서 다중행 비교 연산자와 사용할 수 있음

* 다중행 비교 연산자 : IN, ALL, ANY, ...

```
710 SELECT * FROM EMPLOYEES e
711 WHERE SALARY IN (SELECT MIN(SALARY)
712                  FROM EMPLOYEES e2
713                  GROUP BY DEPARTMENT_ID);
```

Results 1	Results 2	Results 3
* SELECT * FROM EMPLOYEES e WHERE SALARY IN (SELECT MIN(SALARY) FROM EMPLOYEES e2 GROUP BY DEPARTMENT_ID);		
	JOB_ID	SALARY
1	AD_ASST	4,400
2	MK_REP	6,000
3	HR_REP	6,500
4	PR_REP	10,000
5	AC_ACCOUNT	8,300

• 설명

1. 서브쿼리가 실행
2. 부서별 가장 낮은 월급 출력
3. 찾은 결과여러개를 IN에 의해 하나씩 대입하여 조건에 만족하는 데이터 조회
4. 메인 쿼리의 SELECT 실행

```
710 SELECT * FROM EMPLOYEES e
711 WHERE SALARY IN (SELECT MIN(SALARY)
712                  FROM EMPLOYEES e2
713                  GROUP BY DEPARTMENT_ID);
```

Results 1 Results 2 Results 3 ×

*SELECT * FROM EMPLOYEES e WHERE SALARY IN (SELECT MIN(SALARY) FROM EMPLOYEES e2 GROUP BY DEPARTMENT_ID); Enter a SQL expression to filter results (use Ctrl+Space)

		ABC	123	123
		JOB_ID	SALARY	COMM
1	00:00.000	AD_ASST	4,400	
2	00:00.000	MK_REP	6,000	
3	00:00.000	HR_REP	6,500	
4	00:00.000	PR_REP	10,000	
5	00:00.000	AC_ACCOUNT	8,300	

- 설명

결과값이 IN(2100, 4400, 6000, 6500, 10000, ...) 처럼 완성되므로 EMPLOYEES 테이블의 SALARY가 해당 값들이 존재하는 컬럼들의 데이터를 조회함

```
710 SELECT * FROM EMPLOYEES e
711 WHERE SALARY IN (SELECT MIN(SALARY)
712                  FROM EMPLOYEES e2
713                  GROUP BY DEPARTMENT_ID);
```

Results 1 Results 2 Results 3 X

*SELECT * FROM EMPLOYEES e WHERE SALARY IN (SELECT MIN(SALARY) FROM EMPLOYEES e2 GROUP BY DEPARTMENT_ID); Enter a SQL expression to filter results (use Ctrl+Space)

		ABC	123	123
		JOB_ID	SALARY	COMM
1	00:00.000	AD_ASST	4,400	
2	00:00.000	MK_REP	6,000	
3	00:00.000	HR_REP	6,500	
4	00:00.000	PR_REP	10,000	
5	00:00.000	AC_ACCOUNT	8,300	

• 다중행 서브쿼리 사용해보기 - 1

ALL 연산자는 앞에 비교 연산자(>, <, >=, <=)가 붙으며 모두 일치해야 TRUE가 됨

```
724 SELECT * FROM EMPLOYEES e
725 WHERE SALARY > ALL (SELECT MIN(SALARY)
726                     FROM EMPLOYEES e2
727                     GROUP BY DEPARTMENT_ID);
```

Results 1 × Results 2 Results 3 Results 4

SELECT * FROM EMPLOYEES e WHERE SALARY > Enter a SQL expression to filter results (use Ctrl+Space)

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL
1	100	Steven	King	SKING

• 다중행 서브쿼리 사용해보기 - 2

ANY 연산자는 해당하는 값 중 하나라도 일치하면 TRUE를 반환함

```
724 SELECT * FROM EMPLOYEES e
725 WHERE SALARY > ANY (SELECT MIN(SALARY)
726                     FROM EMPLOYEES e2
727                     GROUP BY DEPARTMENT_ID);
```

Results 1	Results 2	Results 3	Results 4
SELECT * FROM EMPLOYEES e WHERE SALARY > ANY (SELECT MIN(SALARY) FROM EMPLOYEES e2 GROUP BY DEPARTMENT_ID);			
EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL
103	James	Landry	JLANDRY
104	Ki	Gee	KGEE
105	Steven	Markle	SMARKLE
106	Hazel	Philtanker	HPHILTAN

• Q2. 실습

1. 직원이 가장 많은 부서의 부서명과 인원수를 조회하세요.
2. Luis 또는 Pat과 같은 부서인 직원들을 조회하세요.
→ 사번(EMPLOYEE_ID), 이름(FIRST_NAME), 부서번호(DEPARTMENT_ID)를 조회하세요.
3. 직무명 FI_ACCOUNT의 평균 월급보다 많이 받는 직원들의 이름과 월급을 조회하세요.
4. 보너스를 받는 직원들의 평균 월급보다 더 높은 월급을 가진 직원들의 이름과 월급을 조회하세요.

• 1번 힌트

아래의 쿼리를 이용하여 가장 많은 부서의 인원을 구할 수 있습니다.

```
SELECT MAX(count(*)) FROM EMPLOYEES e  
GROUP BY DEPARTMENT_ID
```

DML

(다중열 서브쿼리)

• 다중열 서브쿼리

결과값의 컬럼이 여러 개인 서브 쿼리. 단, 데이터(행)은 반드시 하나여야 하며 단일행 비교 연산자와 사용할 수 있음

* 단일행 비교 연산자 : =, <, <=, >=, >, !=

쿼리의 결과가 컬럼은 여러 개여도 되지만, 데이터(행)은 반드시 하나

```
SELECT JOB_ID, DEPARTMENT_ID, SALARY FROM EMPLOYEES e
WHERE (DEPARTMENT_ID, SALARY) = (SELECT DEPARTMENT_ID, SALARY
FROM EMPLOYEES e2
WHERE FIRST_NAME = 'Bruce');
```

Results 1

Results 2

SELECT JOB_ID, DEPARTMENT_ID, SALARY | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC JOB_ID	123 DEPARTMENT_ID	123 SALARY
1	IT_PROG	60	6,000

• 설명

1. 서브쿼리가 실행
2. Bruce 직원의 부서 번호와 월급 조회
3. 찾은 결과들을 각각의 컬럼에 대입하여 조건식 확인
4. 메인 쿼리의 SELECT 실행

```
SELECT JOB_ID, DEPARTMENT_ID, SALARY FROM EMPLOYEES e
WHERE (DEPARTMENT_ID, SALARY) = (SELECT DEPARTMENT_ID, SALARY
FROM EMPLOYEES e2
WHERE FIRST_NAME = 'Bruce');
```

Results 1 Results 2 ×

SELECT JOB_ID, DEPARTMENT_ID, SALARY | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC JOB_ID	123 DEPARTMENT_ID	123 SALARY
1	IT_PROG	60	6,000

- 설명

WHERE (컬럼1, 컬럼2) = (값1, 값2) 와 같이 대입되어 조건식을 계산함

```
SELECT JOB_ID, DEPARTMENT_ID, SALARY FROM EMPLOYEES e  
WHERE (DEPARTMENT_ID, SALARY) = (SELECT DEPARTMENT_ID, SALARY  
FROM EMPLOYEES e2  
WHERE FIRST_NAME = 'Bruce');
```

Results 1

Results 2 X

SELECT JOB_ID, DEPARTMENT_ID, SALARY | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC JOB_ID	123 DEPARTMENT_ID	123 SALARY
1	IT_PROG	60	6,000

• 다중열 서브쿼리 사용해보기 - 1

이메일이 SBELL인 직원과 사수번호, 부서번호가 같은 직원들의 이름 조회하기

```
SELECT FIRST_NAME FROM EMPLOYEES e
WHERE (MANAGER_ID, DEPARTMENT_ID) = (SELECT MANAGER_ID, DEPARTMENT_ID
FROM EMPLOYEES e2
WHERE EMAIL = 'SBELL');
```

Results 1 Results 2 ✕

SELECT FIRST_NAME FROM EMPLOYEES | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC FIRST_NAME
1	Renske
2	Stephen
3	John
4	Joshua
5	Sarah
6	Britney
7	Samuel
8	Vance

• 다중열 서브쿼리 사용해보기 - 2

핸드폰 번호가 650.507.9811인 직원과 같은 부서, 직무를 가지는 직원 조회

```
SELECT EMPLOYEE_ID, FIRST_NAME FROM EMPLOYEES e
WHERE (DEPARTMENT_ID, JOB_ID) = (SELECT DEPARTMENT_ID, JOB_ID
FROM EMPLOYEES e2
WHERE PHONE_NUMBER = '650.507.9811');
```

Results 1 Results 2 ×

SELECT EMPLOYEE_ID, FIRST_NAME FROM | Enter a SQL expression to filter results (use Ctrl+Space)

	123 EMPLOYEE_ID	ABC FIRST_NAME
1	180	Winston
2	181	Jean
3	182	Martha
4	183	Girard
5	184	Nandita
6	185	...

DML

(다중열, 다중행 서브쿼리)

• 다중열, 다중행 서브쿼리

결과값의 컬럼과 데이터(행)가 여러 개인 서브 쿼리. 다중행 비교 연산자와 사용할 수 있음

* 다중행 비교 연산자 : IN, ALL, ANY, ...

```
SELECT FIRST_NAME, LAST_NAME FROM EMPLOYEES e
WHERE (FIRST_NAME, LAST_NAME) IN (SELECT FIRST_NAME, LAST_NAME
FROM EMPLOYEES e2
WHERE SALARY >= 5000);
```

Results 1 Results 2 X

SELECT FIRST_NAME, LAST_NAME FROM I | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC FIRST_NAME	ABC LAST_NAME
1	Ellen	Abel
2	Sundar	Ande
3	Hermann	Baer
4	Amit	Banda
5	Elizabeth	Bates
6	David	Bernstein
7	Harrison	Bloom
8	Gerald	Cambrault

→ 쿼리의 결과가 컬럼 여러 개 가능,
데이터(행) 여러 개 가능

- 설명

WHERE (컬럼1, 컬럼2) IN (값1, 값2) 와 같이 대입되어 조건식을 계산함

```
SELECT FIRST_NAME, LAST_NAME FROM EMPLOYEES e
WHERE (FIRST_NAME, LAST_NAME) IN (SELECT FIRST_NAME, LAST_NAME
FROM EMPLOYEES e2
WHERE SALARY >= 5000);
```

Results 1 Results 2 ×

SELECT FIRST_NAME, LAST_NAME FROM | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC FIRST_NAME	ABC LAST_NAME
1	Ellen	Abel
2	Sundar	Ande
3	Hermann	Baer
4	Amit	Banda
5	Elizabeth	Bates
6	David	Bernstein
7	Harrison	Bloom
8	Gerald	Cambrault

- 다중열, 다중행 서브쿼리 사용해보기 - 1

부서번호가 50번 이면서 이름이 S로 시작하는 직원들의 이름과 연봉 조회하기

```
SELECT FIRST_NAME, SALARY FROM EMPLOYEES e
WHERE (DEPARTMENT_ID, FIRST_NAME) IN (SELECT DEPARTMENT_ID, FIRST_NAME
FROM EMPLOYEES e2
WHERE DEPARTMENT_ID = 50
AND FIRST_NAME LIKE 'S%')
```

Results 1 Results 2 X

SELECT FIRST_NAME, SALARY FROM EMPLOYEES e2 WHERE DEPARTMENT_ID = 50 AND FIRST_NAME LIKE 'S%'

	ABC FIRST_NAME	123 SALARY
1	Sarah	4,000
2	Steven	2,200
3	Samuel	3,200
4	Stephen	3,200
5	Shanta	6,500

• 다중열, 다중행 서브쿼리 사용해보기 - 2

부서별 가장 많은 월급을 받는 직원들의 이름, 부서번호, 월급 조회하기

```
SELECT FIRST_NAME, DEPARTMENT_ID, SALARY  
FROM EMPLOYEES e2  
WHERE (DEPARTMENT_ID, SALARY) IN (SELECT DEPARTMENT_ID, MAX(SALARY)  
                                  FROM EMPLOYEES e3  
                                  GROUP BY DEPARTMENT_ID);
```

Results 1 Results 2 ✕

SELECT FIRST_NAME, DEPARTMENT_ID, SALARY

	ABC FIRST_NAME	123 DEPARTMENT_ID	123 SALARY
1	Steven	90	24,000
2	Alexander	60	9,000
3	Nancy	100	12,008
4	Den	30	11,000
5	Adam	50	8,200
6	John	80	14,000

DML

(인라인 뷰)

• 인라인 뷰

메인쿼리의 FROM에 서브 쿼리를 사용하며, 수행한 결과가 테이블 대신 사용됨

단, 서브쿼리에서 조회한 컬럼들을 기준으로만 메인 쿼리에서 조회할 수 있음

FIRST_NAME 컬럼만 조회 가능

```
SELECT * FROM (SELECT FIRST_NAME FROM EMPLOYEES e
                WHERE E.SALARY >= 10000);
```

Results 1

Results 2

SELECT * FROM (SELECT FIRST_NAME FROM EMPLOYEES e WHERE E.SALARY >= 10000);

	ABC FIRST_NAME
5	Den
6	John
7	Karen
8	Alberto

• 인라인 뷰 서브쿼리 사용해보기 - 1

부서별 가장 많은 월급을 받는 직원 조회하기

```
SELECT * FROM (SELECT DEPARTMENT_ID, MAX(SALARY)
FROM EMPLOYEES e2
GROUP BY DEPARTMENT_ID);
```

Results 1 Results 2 X

SELECT * FROM (SELECT DEPARTMENT_ID | Enter a SQL expression to filter results (use Ctrl+Space)

	123 DEPARTMENT_ID	123 MAX(SALARY)
1	90	24,000
2	60	9,000
3	100	12,008
4	30	11,000

• 인라인 뷰 서브쿼리 사용해보기 - 2

이름이 K로 시작하고 월급을 5000 이상 받는 직원 조회하기

```
SELECT * FROM (SELECT FIRST_NAME FROM EMPLOYEES e
                WHERE FIRST_NAME LIKE 'K%'
                AND SALARY >= 5000);
```

Results 1 Results 2 X

SELECT * FROM (SELECT FIRST_NAME FROM EMPLOYEES e WHERE FIRST_NAME LIKE 'K%' AND SALARY >= 5000); Enter a SQL expression to filter results (use Ctrl+Space)

	ABC FIRST_NAME
1	Kimberely
2	Kevin
3	Karen

• 인라인 뷰를 사용하는 이유

1. 인라인 뷰를 사용할 때는 되도록 조건식을 메인쿼리에 두는것이 좋음
인라인 뷰에서는 필요한 일부 데이터만을 불러오고, 그 데이터에서 조건식을 따지면 성능이 좋아짐 (전체 데이터에서 하나씩 비교해가며 조건식을 보는것보다 빠름)
2. 인라인 뷰를 사용할 때 별칭을 사용하여 쿼리의 가독성을 높일 수도 있음
3. TOP-N 쿼리를 사용하기 위해서 주로 사용됨
 - * TOP-N 쿼리 : 상위 N개의 데이터를 추출하는 쿼리
 - * TOP-N 쿼리는 Q3. 실습 이후 설명

- 참고 1,2번으로 변경된 인라인 뷰

이름이 K로 시작하고 월급을 5000 이상 받는 직원 조회하기

```
SELECT 이름, 월급 FROM (SELECT FIRST_NAME AS 이름,  
                             SALARY AS 월급  
                        FROM EMPLOYEES)  
WHERE 이름 LIKE 'K%'  
AND 월급 >= 5000;
```

Results 1 Results 2 ×

SELECT 이름, 월급 FROM (SELECT FIRST_N | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC 이름 ▼	123 월급 ▼
1	Kimberely	7,000
2	Kevin	5,800
3	Karen	13,500

• 인라인 뷰 서브쿼리 사용해보기 - 3

인라인뷰로 각 부서별 월급의 합계, 평균, 인원수를 구하고 조회하기

```
SELECT E.DEPARTMENT_ID, 합계, 평균, 인원수
FROM (SELECT DEPARTMENT_ID, SUM(SALARY) 합계,
            AVG(SALARY) 평균, COUNT(*) 인원수
      FROM EMPLOYEES
      GROUP BY DEPARTMENT_ID) e
JOIN DEPARTMENTS d
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

Results 1 Results 2 ✕

SELECT E.DEPARTMENT_ID, 합계, 평균, 인 · Enter a SQL expression to filter results (use Ctrl+Space)

	123 DEPARTMENT_ID	123 합계	123 평균	123 인원수
1	90	58,000	19,333.3333333333	3
2	60	28,800	5,760	5
3	100	51,608	8,601.3333333333	6
4	30	24,900	4,150	6
5	50	156,400	3,475.5555555556	45
6	80	304,500	8,955.8823529412	34
7	10	4,400	4,400	1
8	20	19,000	9,500	2
9	40	6,500	6,500	1
10	70	10,000	10,000	1
11	110	20,308	10,154	2

• Q3. 실습

1. 가장 월급이 높은 직원의 이름과 월급을 조회하세요
2. JOBS 테이블에서 JOB_TITLE별로 MAX_SALARY를 확인하고, 가장 MAX_SALARY가 높은 JOB_TITLE를 조회하세요.
3. 부서별로 평균 연봉을 계산하고, 평균 연봉이 가장 높은 부서의 이름과 평균 연봉을 조회하세요.

- TOP-N 쿼리

상위 N개의 데이터를 추출하기 위한 쿼리

ex) 직원중 급여가 가장 높은 상위 3명, 입사한지 가장 오래된 직원 상위 5명, ...

- 종류

1. ROWNUM

→ 오라클에서 사용되며, WHERE에서 행의 개수를 제한함

→ 단, ROWNUM으로 TOP-N 쿼리를 만들기 위해서는 인라인뷰 사용 필요

2. ROW LIMITING

→ ANSI 표준 문법

→ 그동안 실습에서 자주 사용했던 'FETCH FIRST 5 ROWS ONLY'

→ 실습에서 사용한 것 외에도 행의 개수를 제한하는 옵션들이 있음

3. 기타

- ROWNUM 사용해보기 - 1

월급이 높은 직원 3명 조회하기

인라인뷰에서 정렬 후 ROWNUM <= 3을 통해 3번째 행보다 낮거나 같은 데이터만 출력

```
SELECT *  
FROM (SELECT FIRST_NAME, EMAIL  
      FROM EMPLOYEES e  
      ORDER BY SALARY DESC)  
WHERE ROWNUM <= 3;
```

Results 1 Results 2 ✕

* SELECT * FROM (SELECT FIRST_NAME, EM | Enter a SQL expression to filter

	ABC FIRST_NAME ▼	ABC EMAIL ▼
1	Steven	SKING
2	Neena	NKOCHHAR
3	Lex	LDEHAAN

- 참고

메인쿼리 컬럼에 ROWNUM을 작성하여 몇번째 행인지 컬럼을 추가하여 데이터를 조회할 수 있음

```
SELECT ROWNUM, FIRST_NAME, EMAIL  
FROM (SELECT FIRST_NAME, EMAIL  
      FROM EMPLOYEES e  
      ORDER BY SALARY DESC)  
WHERE ROWNUM <= 3;
```

	ROWNUM	FIRST_NAME	EMAIL
1	1	Steven	SKING
2	2	Neena	NKOCHHAR
3	3	Lex	LDEHAAN

- ROWNUM 사용해보기 - 2

입사한지 가장 오래된 직원 상위 5명 조회하기

```
SELECT ROWNUM, FIRST_NAME, EMAIL  
FROM (SELECT FIRST_NAME, EMAIL  
      FROM EMPLOYEES e  
      ORDER BY HIRE_DATE ASC)  
WHERE ROWNUM <= 5;
```

Results 1 Results 2 ×

SELECT ROWNUM, FIRST_NAME, EMAIL FROM EMPLOYEES ORDER BY HIRE_DATE ASC

	ROWNUM	FIRST_NAME	EMAIL
1	1	Lex	LDEHAAN
2	2	Susan	SMAVRIS
3	3	William	WGNETZ
4	4	Shelley	SHIGGINS
5	5	Hermann	HBAER

• Q4. 실습

1. 월급이 6000 이상인 직원 중에서 상위 5명의 직원 이름과 연봉을 조회하세요.
2. JOBS 테이블에서 JOB_TITLE을 ABC순으로 정렬하고 상위 10개의 JOB_TITLE을 조회하세요.
3. DEPARTMENT 테이블에서 DEPARTMENT_ID를 기준으로 내림차순 정렬하여 상위 15개의 ID, NAME, LOCATION_ID를 조회하세요.
4. 이름이 e로 끝나는 직원 중 입사일이 오래된 상위 3명의 직원 이름과 연봉, 입사일을 조회하세요.
5. 전화번호가 4로 끝나는 직원 중 이메일을 ABC순으로 정렬하고 상위 3명의 직원 이름과 이메일을 조회하세요.
6. 보너스를 가진 직원들의 보너스를 포함한 연봉(*12)을 계산하고 가장 많은 연봉을 받는 상위 5명의 직원 이름과 보너스를 포함한 연봉을 조회하세요.
 - 단, 첫번째 컬럼의 별칭을 "순위"로 만들고 조회합니다.
 - 직원 이름의 별칭을 "이름"으로 만들고 조회합니다.
 - 보너스를 포함한 연봉의 별칭을 "보너스 연봉"으로 만들고 조회합니다.